

VAE、Diffusion、Flow Matching 系统讲解（一）：VAE 变分自编码器



冰锐
吉大车辆工程

[关注他](#)

收录于 · 我的无人驾驶学习历程 ›

2 人赞同了该文章 ›

【系列文章导航】VAE、Diffusion 与 Flow Matching 系统性深度讲解

- （一）VAE 变分自编码器（本篇）
- （二）Diffusion 扩散模型完全指南
- （三）Flow Matching 流匹配
- （四）三者对比、Stable Diffusion/DiT 与进阶话题

VAE、Diffusion、Flow Matching 系统讲解（一）：VAE 变分自编码器

三种核心生成模型范式的原理、数学推导、本质理解与难点剖析

目录

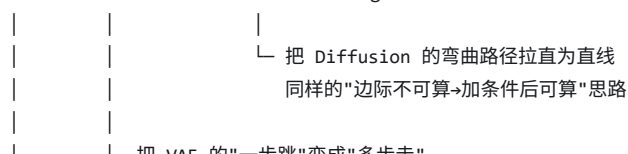
1. VAE（变分自编码器）
2. Diffusion Models（扩散模型）
3. Flow Matching（流匹配）
4. 三者对比与统一视角
5. 在 Stable Diffusion 中的角色（含 \$5.3 DiT 架构*详解）
6. 进阶话题

全局导读地图

阅读建议：本文三章内容较长，但核心逻辑链非常清晰。下面的地图帮你快速定位**必读**的核心推导、可以跳读的细节、以及三章之间的递进关系。

三章的递进关系

VAE → Diffusion → Flow Matching



已赞同 2 ▼ 1 条评论 ★ 2 ♡ 喜欢 ➔ 分享 📄 申请转载 ✨ ...

关于作者



冰锐
吉大车辆工程

回答

7

文章

48

关注者

3,597

[关注他](#)[发私信](#)

大家都在搜

换一换

- 美国白宫记者晚宴发生枪... 367 万 新
- 男子性情大变确诊神经梅... 366 万 热
- DeepSeek V4 预览版上线... 333 万
- AC 米兰国米球员被曝集体... 310 万
- 华谊兄弟被正式申请破产 297 万
- 爱喝无糖饮料的天塌了 297 万
- 东方甄选4名主播集体离职 283 万 新
- OpenAI 发布 GPT-5.5 270 万
- 男童疑被误诊为脑瘫治疗 7... 258 万
- 张军 253 万



核心逻辑链（三章共享的母题）

三章面对的都是**同一个根本困难**，用的是**同一类解法**：

	困难	解法	网络学什么
VAE	$p(z x)$ 后验算不出来	用 Encoder $q_{\varphi}(z x)$ 近似	近似后验的参数 μ, σ
Diffusion	$q(x_{t-1} x_t)$ 反向后验算不出来	给定 x_0 后可算 $\rightarrow$ 训练网络预测 ϵ	噪声 ϵ （等价于 score）
Flow Matching	边际速度场 $u_t(x)$ 算不出来	给定 (x_0, x_1) 后可算 $\rightarrow$ 训练网络预测 v	速度 $v = x_1 - x_0$

一句话：都是“**边际不可算 $\rightarrow$ 加条件后可算 $\rightarrow$ 用网络学那个可算的量**”。

必读推导 vs 可跳读细节

标记说明：[核心] = 核心推导，必须理解逻辑链；[记结论] = 重要但可先跳过细节、记结论即可；[了解] = 直觉理解/补充知识

章节	重要性	说明
1.3 ELBO 推导	[核心]	VAE 的数学核心，后面 Diffusion 的 VLB 是它的推广
1.4 重参数化 ⁺	[记结论]	记住 $z = \mu + \sigma \cdot \epsilon$ 即可，Diffusion 中还会反复用到
1.5 KL 闭式解	[了解]	记住最终公式就行，推导过程可跳
1.6 VAE 困境	[记结论]	理解“为什么 VAE 生成模糊”帮助理解为什么需要 Diffusion
2.2 前向过程闭式解	[核心]	$x_t = \sqrt{\alpha_t} \cdot x_0 + \sqrt{(1-\alpha_t)} \cdot \epsilon$ 是后面一切的基础
2.3 反向过程推导	[核心++]	全文最核心的推导。从“为什么不可算”到“配方得闭式解”到“用 ϵ 替换 x_0 ”，这条逻辑链必须走通
2.4 损失函数推导	[核心]	从 VLB $\rightarrow$ KL $\rightarrow$ MSE 的简化链
2.5 三种预测目标	[记结论]	记住“ ϵ 、 x_0 、 v 三者等价”即可
2.6 DDPM vs DDIM	[记结论]	理解“SDE vs ODE”、“增量 vs 坐标”两个直觉
2.7 Score Matching	[核心]	连接 Diffusion 和 Flow Matching 的桥梁
3.2 CNF 数学基础	[了解]	流映射和连续性方程的直觉理解即可
3.3-3.4 CFM 核心突破	[核心++]	Flow Matching 的核心：为什么条件目标等价于边际目标
3.4.5 CFM=FM 证明	[记结论]	结论比过程重要——“L2 损失自动学到平均”是核心直

已赞同 2 1 条评论 2 喜欢 分享 申请转载 ...

3.8 OT-CFM	[了解]	优化技巧，理解“路径交叉问题”的直觉即可
5.3 DiT 架构	[核心]	Diffusion/FM 的现代骨干——adaLN-Zero + Scaling Law, Sora/SD3 的基础
6.1 条件引导 (CG→CFG)	[核心]	所有文生图模型的关键技术，理解 CFG 公式和 w 的含义
6.2 Rectified Flow	[记结论]	理解“Reflow 拉直路径 → 1-2 步采样”的直觉即可
6.3 Consistency Models	[了解]	一步生成的另一种思路，理解“自治性约束”的直觉

一遍读完后，最值得记住的 6 件事

- ELBO 思想**：真实目标不可算 → 优化一个可算的下界 (VAE/Diffusion 共用)
- Diffusion 的反向过程推导链**： $q(x_{t-1}|x_t)$ 不可算 → 给定 x_0 后可算 → 用 ϵ 替换 x_0 → 训练网络预测 ϵ
- 预测噪声 = 学 score**： $\epsilon_{\theta} \approx -\sqrt{1-\bar{\alpha}_t} \nabla_{x_t} \log q(x_t)$
- Flow Matching 的简化**：直线路径 $(1-t)x_0 + tx_1$ 、常数速度 $x_1 - x_0$ 、不需要噪声调度
- L2 损失的魔法**：训练个体目标（单个 ϵ 或单个 $x_1 - x_0$ ），自动学到平均（边际速度场 / score）
- CFG 引导**： $\tilde{v} = v^{\text{uncond}} + w \cdot (v^{\text{cond}} - v^{\text{uncond}})$ ——用条件与无条件预测的差来引导生成方向， w 控制引导强度

1. VAE（变分自编码器）

1.1 核心思想

生成模型的目标：学会数据分布 $p(x)$ ，从而能采样生成新数据。

VAE 的策略：假设数据由低维隐变量 z 生成（先采样 z ，再经 Decoder 映射成 x ）：

$$p(x) = \int p(x|z)p(z)dz$$

为什么假设“隐变量生成数据”？ 这基于**流形假设**：高维数据的实际自由度远低于维度。例如一张 28×28 的手写数字（784 维），其变化因素只有笔画粗细、倾斜角度、大小、风格等约 10-20 个因素。VAE 用一个低维向量 z （如 20 维）来表示这些潜在因素，训练一个 Decoder 网络把 z 映射成图片 x 。生成时只需从简单分布采样 z ，再通过 Decoder 就能得到新数据。

问题 1： $p(x)$ 不可直接计算

$p(x) = \int p(x|z)p(z)dz$ 需要对所有可能的 z 求积分。 z 通常是 20-512 维的连续向量， $p(x|z)$ 是复杂的神经网络输出——在高维空间穷举所有 z 是不可行的。

问题 2：训练还需要后验 $p(z|x)$ ，但它也算不出来

为什么训练需要 $p(z|x)$ ？ 训练 Decoder 时，你有一张训练图 x ，想让 Decoder 学会重建

已赞同 2 1 条评论 2 喜欢 分享 申请转载 ...

但 $p(z|x) = \frac{p(x|z)p(z)}{p(x)}$ ，分母 $p(x)$ 就是上面那个不可解的积分。于是后验也算不出来。

VAE 的解法——变分推断：既然真实后验 $p(z|x)$ 算不出来，就训练一个 **Encoder 网络** $q_\phi(z|x)$ （输出高斯分布的参数 μ, σ ）去近似它。这样 Encoder 就能为每个训练数据 x “推断”出合适的 z ，送给 Decoder 去重建。然后优化一个可计算的下界 ELBO 来替代不可解的积分：

$$\log p(x) = \underbrace{\text{ELBO}}_{\text{可优化}} + \underbrace{D_{\text{KL}}(q_\phi(z|x) \parallel p(z|x))}_{\geq 0}$$

最大化 ELBO 同时做到两件事：让生成质量更好（提升 $\log p(x)$ ）、让 Encoder 近似更准（压缩 KL 间隙）。

“变分”是什么意思？ “变分”来自数学中的**变分法**（Calculus of Variations）——在一族函数中找最优的那个。VAE 中的“变分推断”指的是：定义一个参数化的分布族 $\{q_\phi(z|x)\}$ （如对角高斯），然后在这个族中**搜索**最接近真实后验 $p(z|x)$ 的那个分布。也就是说，把“精确计算后验”（需要积分，不可行）转化为“在分布族中搜索最优近似”（优化问题，可行）。代价是引入了近似误差—— q_ϕ 永远不可能完美等于 $p(z|x)$ ，这也是 VAE 生成质量不如 Diffusion 的原因之一。

这个公式现在不需要完全理解，1.3 节会一步步推导。这里只需要记住核心思路：**真实后验 $p(z|x)$ 算不出来 → 用网络 $q_\phi(z|x)$ 近似 → 优化 ELBO 作为替代目标。**

本章逻辑路线图（建议先浏览，再逐节精读）：

- 1.1-1.2 核心思想和架构**：数据由隐变量 z 生成，Encoder 编码分布、Decoder 解码重建
- 1.3 ELBO 推导** ← **核心数学** → 出发点：想最大化 $\log p(x)$ ，但积分不可解 → 引入近似后验 q_ϕ + **Jensen 不等式** → 得到可优化的下界 ELBO → ELBO = 重建项（Decoder 重建得好不好）- KL 项（Encoder 偏离先验多少）
- 1.4 重参数化技巧**：采样操作不可微怎么办？→ $z = \mu + \sigma \cdot \epsilon$ ，把随机性外包给 ϵ
- 1.5 KL 散度** 闭式解：两个高斯之间的 KL 有精确公式（不需要蒙特卡洛估计）
- 1.6 VAE 的困境**：重建 vs 正则化的矛盾 → 生成模糊 / 后验坍塌

1.2 架构

$$\begin{array}{c} x \longrightarrow [\text{Encoder } q_\phi(z|x)] \longrightarrow \mu, \sigma^2 \\ \downarrow \\ z = \mu + \sigma \cdot \epsilon \quad (\epsilon \sim N(0, I), \text{ 重参数化}) \\ \downarrow \\ z \longrightarrow [\text{Decoder } p_\theta(x|z)] \longrightarrow \hat{x} \text{ (重建)} \end{array}$$

- Encoder $q_\phi(z|x)$** ：数据 $\rightarrow$ 隐变量分布的参数（均值 μ 、方差 σ^2 ）
- Decoder $p_\theta(x|z)$** ：隐变量 $\rightarrow$ 重建数据

1.3 ELBO 的完整推导——损失函数是怎么来的？

本节是 VAE 的数学核心。如果你只记住一件事：VAE 的损失函数不是随意定义的，而是从“最大化数据的对数似然”这个目标出发，经过严格的数学推导得到的。

1.3.1 出发点：最大化数据的对数似然

我们希望最大化 $\log p_\theta(x)$ 。但直接计算 $p_\theta(x) = \int p_\theta(x|z)p(z)dz$ 不可行（1.1 节已解释原因）。

「优胜王」件。这是概率模型训练的标准做法（取近似似然估计，MLE）。

核心问题：目标函数里有一个算不出来的积分。怎么办？→ 用一个**可计算的下界**来替代它。

1.3.2 引入变分分布

这一步的动机：积分算不出来，是因为我们不知道对哪些 z 积分是“有用的”。如果有一个分布 $q_{\phi}(z|x)$ 能告诉我们“对于给定的 x ，哪些 z 最重要”，就可以用它来做重要性采样，把积分变成期望。

引入一个近似后验 $q_{\phi}(z|x)$ （由 Encoder 参数化），对 $\log p_{\theta}(x)$ 做如下变换：

$$\log p_{\theta}(x) = \log \int p_{\theta}(x, z) \, dz$$

利用 $q_{\phi}(z|x)$ 做重要性采样——乘一个 $\frac{q_{\phi}(z|x)}{q_{\phi}(z|x)} = 1$ ，把积分变成期望：

$$\log p_{\theta}(x) = \log \int \frac{p_{\theta}(x, z)}{q_{\phi}(z|x)} \cdot q_{\phi}(z|x) \, dz = \log \mathbb{E}_{q_{\phi}(z|x)} \left[\frac{p_{\theta}(x, z)}{q_{\phi}(z|x)} \right]$$

到此为止还是等式，没有任何近似。但 $\log$ 里面套了一个期望，还是不好算。下一步用 Jensen 不等式把 $\log$ 推进期望里面。

1.3.3 Jensen 不等式下推

由于 $\log$ 是凹函数，根据 Jensen 不等式 $\log \mathbb{E}[X] \geq \mathbb{E}[\log X]$ ：

$$\log p_{\theta}(x) \geq \mathbb{E}_{q_{\phi}(z|x)} \left[\log \frac{p_{\theta}(x, z)}{q_{\phi}(z|x)} \right]$$

这个下界就是 **ELBO (Evidence Lower Bound, 证据下界)**：

$$\text{ELBO} = \mathbb{E}_{q_{\phi}(z|x)} \left[\log \frac{p_{\theta}(x, z)}{q_{\phi}(z|x)} \right]$$

为什么叫“证据下界”？ “Evidence” 在贝叶斯统计中指观测数据的似然 $p(x)$ ，也叫 marginal likelihood。ELBO 是 $\log p(x)$ 的一个下界，所以叫 Evidence Lower Bound。

1.3.4 ELBO 的分解——直觉形式

目标：把 ELBO 分解成可直理解的两项，看看优化 ELBO 到底在做什么。

把联合概率拆开 $p_{\theta}(x, z) = p_{\theta}(x|z) \cdot p(z)$ ：

$$\text{ELBO} = \mathbb{E}_{q_{\phi}(z|x)} \left[\log p_{\theta}(x|z) + \log p(z) - \log q_{\phi}(z|x) \right]$$

$$= \mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x|z)] - D_{\text{KL}}(q_{\phi}(z|x) \parallel p(z))$$

（第一项为重建项，第二项为正则项。）

其中 KL 散度的展开用了定义 $D_{\text{KL}}(q \parallel p) = \mathbb{E}_q[\log q - \log p]$ 。

1.3.5 ELBO 与真实对数似然的关系

精确的关系是：

$$\begin{aligned} D_{\text{KL}}(q_{\phi}(z|x) \parallel p_{\theta}(z|x)) &= \mathbb{E}_{q_{\phi}} \left[\log \frac{q_{\phi}(z|x)}{p_{\theta}(z|x)} \right] \\ &= \mathbb{E}_{q_{\phi}} \left[\log q_{\phi}(z|x) - \log p_{\theta}(z|x) \right] \\ &= \mathbb{E}_{q_{\phi}} \left[\log q_{\phi}(z|x) - \log p_{\theta}(x,z) + \log p_{\theta}(x) \right] \\ &= \mathbb{E}_{q_{\phi}} \left[\log q_{\phi}(z|x) - \log p_{\theta}(x,z) \right] + \log p_{\theta}(x) \end{aligned}$$

因为 $D_{\text{KL}} \geq 0$ ，所以 $\log p_{\theta}(x) \geq \text{ELBO}$ ，ELBO 确实是下界。

关键洞察：最大化 ELBO 同时做了两件事：

1. **最大化数据似然** $\log p_{\theta}(x)$ （让生成模型变好）
2. **最小化近似间隙** $D_{\text{KL}}(q_{\phi} \parallel p_{\theta}(z|x))$ （让 Encoder 的近似更准确）

1.3.6 VAE 损失函数的最终形式

取负号（最大化 ELBO $\rightarrow$ 最小化负 ELBO）：

$$\mathcal{L}_{\text{VAE}} = -\mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x|z)] + D_{\text{KL}}(q_{\phi}(z|x) \parallel p(z))$$

项	含义	直觉
重建损失 $-\mathbb{E}[\log p_{\theta}(x z)]$	Decoder 从 z 重建 x 的负对数似然	如果 Decoder 输出高斯分布，就是 MSE
KL 正则项 $D_{\text{KL}}(q \parallel p)$	Encoder 输出的分布偏离先验 $N(0, I)$ 的程度	让隐空间平滑可采样

为什么重建损失是 MSE？ 如果假设 $p_{\theta}(x|z) = \mathcal{N}(x; \mu_{\theta}(z), \sigma^2 I)$ ，那么：

$$-\log p_{\theta}(x|z) = \frac{\|x - \mu_{\theta}(z)\|^2}{2\sigma^2} + \text{const}$$

忽略常数和 σ^2 缩放后，就是均方误差 (MSE)。

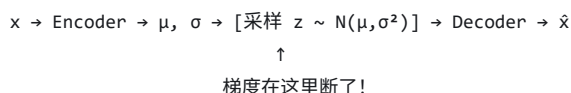
1.4 重参数化技巧——为什么需要？为什么能工作？

1.4.1 问题：采样操作不可微

训练 VAE 需要反向传播计算梯度。但 ELBO 中有一项 $\mathbb{E}_{q_{\phi}(z|x)} [\dots]$ ，意味着我们需要从 $q_{\phi}(z|x) = \mathcal{N}(\mu_{\phi}(x), \sigma^2_{\phi}(x))$ 中**采样** z 。

问题在于：“**从分布中采样**”这个操作本身是**不可微的**。

想象反向传播：梯度需要从 Decoder 流回 Encoder。但中间有一步 $z \sim \mathcal{N}(\mu, \sigma^2)$ (μ, σ^2)，这是一个随机操作，梯度无法穿过它。



1.4.2 解决方案：把随机性“外包”

核心思想：把随机性从参数中分离出来。

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

$$\begin{array}{c}
 x \rightarrow \text{Encoder} \rightarrow \mu, \sigma \rightarrow z = \mu + \sigma \cdot \epsilon \rightarrow \text{Decoder} \rightarrow \hat{x} \\
 \uparrow \\
 \epsilon \sim N(0, I) \text{ (与参数无关, 梯度可以正常流过)}
 \end{array}$$

1.4.3 数学上为什么等价？

这利用了概率论中的一个基本性质：如果 $\epsilon \sim \mathcal{N}(0, I)$ ，那么 $z = \mu + \sigma \cdot \epsilon$ 的分布恰好是 $\mathcal{N}(\mu, \sigma^2 I)$ 。

所以 $\mathbb{E}_{z \sim q_\phi}[f(z)] = \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)}[f(\mu + \sigma \cdot \epsilon)]$

后者对 μ 和 σ 可以直接用蒙特卡洛估计来计算梯度（因为期望是对固定分布 $\mathcal{N}(0, I)$ 取的，不依赖于 ϕ ）。

1.4.4 更深层的理解

重参数化技巧并非 VAE 独有，它是**随机计算图**中的通用方法。核心原则是：

想对一个随机变量的分布参数求梯度时，设法把随机变量写成“确定性函数 + 外部噪声”的形式。

这在后续的 Diffusion 中也被大量使用（前向过程中的 $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ 本质上也是重参数化）。

1.5 KL 散度的闭式解推导

当 $q(z|x) = \mathcal{N}(\mu, \text{diag}(\sigma^2))$ ， $p(z) = \mathcal{N}(0, I)$ 时，KL 散度有闭式解。

推导过程（以一维为例，多维逐维独立求和）：

$$\begin{aligned}
 D_{\text{KL}} &= \mathbb{E}_q \left[\log \frac{q(z)}{p(z)} \right] \\
 &= \mathbb{E}_q [\log q(z) - \log p(z)]
 \end{aligned}$$

对于一维高斯：

- $\log q(z) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{(z-\mu)^2}{2\sigma^2}$
- $\log p(z) = -\frac{1}{2} \log(2\pi) - \frac{z^2}{2}$

代入：

$$\begin{aligned}
 D_{\text{KL}} &= \mathbb{E}_q \left[-\frac{1}{2} \log \sigma^2 - \frac{(z-\mu)^2}{2\sigma^2} + \frac{z^2}{2} \right]
 \end{aligned}$$

逐项计算期望：

- $\mathbb{E}_q \left[-\frac{1}{2} \log \sigma^2 \right] = -\frac{1}{2} \log \sigma^2$ （常数，直接提出）
- $\mathbb{E}_q \left[-\frac{(z-\mu)^2}{2\sigma^2} \right] = -\frac{1}{2\sigma^2} \mathbb{E}_q [(z-\mu)^2] = -\frac{1}{2\sigma^2} \text{Var}(z) = -\frac{1}{2}$ （因为 $\text{Var}(z) = \sigma^2$ ）
- $\mathbb{E}_q \left[\frac{z^2}{2} \right] = \frac{1}{2} \mathbb{E}_q [z^2]$

所以：

$$D_{\text{KL}} = -\frac{1}{2} \log \sigma^2 - \frac{1}{2} + \frac{1}{2} (\mu^2 + \sigma^2) \\ = \frac{1}{2} \left(-\log \sigma^2 - 1 + \mu^2 + \sigma^2 \right)$$

推广到 d 维（各维独立）：

$$D_{\text{KL}} = -\frac{1}{2} \sum_{j=1}^d \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2 \right)$$

为什么这个闭式解很重要？ 因为它让我们不需要用蒙特卡洛采样去估计 KL 散度，直接用 μ 和 σ 就能精确计算，避免了估计带来的方差。

1.6 VAE 的本质困境与后验坍缩

1.6.1 重建 vs 正则化的张力

两项损失之间存在根本性矛盾：

- **重建项**想让 Encoder 把尽可能多的信息塞进 $z \rightarrow \sigma$ 越小越好（精确传递信息）
- **KL 项**想让 $q(z|x) \rightarrow \mathcal{N}(0, I) \rightarrow \mu \rightarrow 0, \sigma \rightarrow 1$ （丢弃信息）

这种张力导致 VAE 生成的图像往往偏模糊——它是所有可能重建的“平均”。

1.6.2 后验坍缩（Posterior Collapse）

当 Decoder 太强大时（如自回归 Decoder），它可以忽略 z 直接从上下文生成 x 。此时 Encoder 的 KL 项被优化到 0， $q(z|x) \approx p(z)$ ， z 不再携带任何信息。

缓解方法：

- KL 退火（训练初期降低 KL 权重）
- 使用 β -VAE（手动调节 KL 权重 β ）
- Free bits（给每维 KL 设最小值）

1.7 特点

- [+] 训练快（一次前传，无需迭代采样）
- [+] 隐空间有明确结构，可做插值、属性编辑
- [+] 可以做数据压缩（作为其他模型的前端）
- [-] 生成质量一般（后验近似 + 重建目标导致模糊）
- [-] 表达能力受限于高斯假设

1.8 应用

- **Stable Diffusion 的图像编解码器**（KL-VAE，把 512×512 图压缩到 64×64 隐空间）
- 异常检测、数据增强、半监督学习

1.9 核心速记卡

看完 VAE 后，闭上文档能说出这些，就算掌握了。

一句话总结： 真实后验 $p(z|x)$ 算不出来 $\rightarrow$ 用 Encoder $q_\phi(z|x)$ 近似 $\rightarrow$ 优化 ELBO（重建 + KL）作为替代目标。

核心逻辑链（5 步）：

已赞同 2 1 条评论 2 喜欢 分享 申请转载 ...

3. ELBO = 重建项 (Decoder 重建得好不好) - KL 项 (Encoder 偏离先验多少)
- ↓
4. 采样 z 不可微 $\rightarrow$ 重参数化: $z = \mu + \sigma \cdot \epsilon$, 梯度可以流过
- ↓
5. 训练 Encoder + Decoder, 最小化 -ELBO

关键公式 (3 个) :

公式	含义
$\log p(x) = \text{ELBO} + D_{\text{KL}}(q_{\varphi} \parallel p(z x))$	ELBO 是对数似然的下界
$L = -E[\log p_{\theta}(x z)] + D_{\text{KL}}(q_{\varphi} \parallel p(z))$	损失 = 重建 + KL
$z = \mu + \sigma \epsilon, \epsilon \sim N(0, I)$	重参数化技巧

和下一章的衔接: VAE 一步跳太远 (从 z 直接到 x) , 导致模糊。Diffusion 的策略是” 把一步大跳拆成多步小走” —— 逐步去噪, 每步只需做微小调整, 精度更高。

1.10 PyTorch 实现

```
import torch
import torch.nn as nn
import torch.nn.functional as F

# =====
# Part 1: VAE 模型 – Encoder + 重参数化 + Decoder
# =====

class VAE(nn.Module):
    """
    变分自编码器, 实现 1.2 节的架构:
     $x \rightarrow \text{Encoder} \rightarrow (\mu, \log \sigma^2) \rightarrow \text{重参数化} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$ 

    和 Diffusion/Flow Matching 的关键区别:
    - VAE 是"一步到位":  $z \rightarrow \hat{x}$  只需一次前向传播
    - Diffusion/FM 需要多步迭代 (逐步去噪 / ODE 积分)
    """
    def __init__(self, input_dim=784, hidden_dim=400, latent_dim=20):
        super().__init__()
        # Encoder:  $x \rightarrow (\mu, \log \sigma^2)$ 
        self.enc_fc1 = nn.Linear(input_dim, hidden_dim)
        self.enc_mu = nn.Linear(hidden_dim, latent_dim)
        self.enc_logvar = nn.Linear(hidden_dim, latent_dim)
        # Decoder:  $z \rightarrow \hat{x}$ 
        self.dec_fc1 = nn.Linear(latent_dim, hidden_dim)
        self.dec_out = nn.Linear(hidden_dim, input_dim)

    def encode(self, x):
        """ $x \rightarrow$  隐变量分布的参数  $(\mu, \log \sigma^2)$ """
        h = F.relu(self.enc_fc1(x))
        return self.enc_mu(h), self.enc_logvar(h)
```

```
"""
    """
    std = torch.exp(0.5 * logvar) #  $\sigma = \exp(0.5 * \log \sigma^2)$ 
    eps = torch.randn_like(std)    #  $\epsilon \sim N(0, I)$ , 与参数无关
    return mu + std * eps          #  $z$  对  $\mu, \sigma$  可微

def decode(self, z):
    """ $z \rightarrow$  重建数据  $\hat{x}$ """
    h = F.relu(self.dec_fc1(z))
    return torch.sigmoid(self.dec_out(h))

def forward(self, x):
    """
    输入: x (B, input_dim) - 展平后的图像
    输出: x_recon (B, input_dim), mu (B, latent_dim), logvar (B, latent_dim)
    """

    # Encoder: 推断隐变量分布
    mu, logvar = self.encode(x)
    # 重参数化采样
    z = self.reparameterize(mu, logvar)
    # Decoder: 重建
    x_recon = self.decode(z)
    return x_recon, mu, logvar

# =====
# Part 2: ELBO 损失函数
# =====

def vae_loss(x_recon, x, mu, logvar):
    """
    ELBO 损失 (1.3.6 节) =  $-E[\log p(x|z)] + KL(q(z|x) || p(z))$ 

    - 重建项: 二值交叉熵 (假设 Bernoulli decoder) 或 MSE (假设 Gaussian decoder)
    - KL 项: 闭式解 (1.5 节推导)
    """
    # 重建损失:  $-E[\log p(x|z)]$ 
    recon_loss = F.binary_cross_entropy(x_recon, x, reduction='sum')

    # KL 散度闭式解:  $-0.5 * \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2)$ 
    kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())

    return recon_loss + kl_loss

# =====
# Part 3: 训练循环与使用示例
# =====

def train_vae(model, dataloader, optimizer, device, epochs=10):
    """标准训练循环。MNIST 上训练 10-20 个 epoch 即可看到效果。"""
    model.train()
    for epoch in range(epochs):
        total_loss = 0
        for x_batch, _ in dataloader:
            x_batch = x_batch.view(-1, 784).to(device)

            x_recon, mu, logvar = model(x_batch)
            loss = vae_loss(x_recon, x_batch, mu, logvar)
```

已赞同 2



1 条评论

★ 2

♥ 喜欢

➔ 分享

📄 申请转载



...

```
optimizer.zero_grad_()
total_loss += loss.item()

avg_loss = total_loss / len(dataloader.dataset)
if (epoch + 1) % 5 == 0:
    print(f"Epoch {epoch+1}/{epochs}, Loss: {avg_loss:.4f}")

@torch.no_grad()
def sample_vae(model, num_samples=16, latent_dim=20, device='cpu'):
    """
    从先验  $N(0, I)$  采样  $z$ ，一步解码生成新数据。
    对比 Diffusion/FM：它们需要多步迭代，VAE 只需一步。
    """
    z = torch.randn(num_samples, latent_dim, device=device)
    samples = model.decode(z)
    return samples

# ===== 完整使用示例（以 MNIST 为例） =====
if __name__ == "__main__":
    from torchvision import datasets, transforms
    from torch.utils.data import DataLoader

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # 1. 数据：MNIST 28x28 灰度图
    transform = transforms.Compose([transforms.ToTensor()])
    dataset = datasets.MNIST(root="./data", train=True, download=True, transform=
    dataloader = DataLoader(dataset, batch_size=128, shuffle=True)

    # 2. 模型
    model = VAE(input_dim=784, hidden_dim=400, latent_dim=20).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

    # 3. 训练
    train_vae(model, dataloader, optimizer, device, epochs=20)

    # 4. 采样（一步解码，无需迭代）
    model.eval()
    samples = sample_vae(model, num_samples=16, latent_dim=20, device=device)
    print(f"Samples shape: {samples.shape}") # (16, 784)
```

代码与公式的对应关系：

代码	对应的数学公式	文档章节
<code>mu + std * eps</code>	$z = \mu + \sigma \epsilon$	1.4 重参数化
<code>binary_cross_entropy</code>	$-E[\log p(x z)]$	1.3.6 重建项
<code>-0.5 * sum(1 + logvar - mu² - exp(logvar))</code>	$-\frac{1}{2} \sum (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$	1.5 KL 闭式解
<code>torch.randn</code> → <code>decode</code>	从 $p(z)=N(0, I)$ 采样后解码	1.8 生成过程

本章小结

- 核心问题：** $p(x) = \int p(x|z)p(z)dz$ 中的积分不可算 → 引入近似后验 $q_\phi(z|x)$ → 优化 ELBO 下界

已赞同 2

1 条评论

★ 2

♥ 喜欢

➔ 分享

📄 申请转载

✦ ...

下一篇预告

（二）Diffusion 扩散模型完全指南：从前向加噪到 Score Matching 将系统讲解前向加噪与反向去噪的马尔可夫链、VLB/ELBO、 ϵ 预测与 score 的关系、DDPM/DDIM 与三种预测目标等，把你在这篇里学的 ELBO 与重参数化，接到「多步去噪」的主线上。

送礼物

还没有人送礼物，鼓励一下作者吧

所属专栏 · 12 小时前 更新



我的无人驾驶学习历程

冰锐

43 篇内容 · 3862 赞同

订阅

最热内容 · 985 车辆工程想往无人驾驶，智能车，车联网方向发展，大学期间应该具体自学哪些课程 and 知识？

编辑于 2026-04-25 22:58 · 北京

Diffusion



别再撸电子猫了，下一代“电子龙虾”搭子 DuMate...

AI 兴趣党的快乐，就是有个能替你熬夜的龙虾级搭子。 [查看详情](#)

百度智能云 的广告



理性发言，友善互动

1 条评论

默认

最新



欧耶 奥奥的木泥

nb 最清晰易懂的一集👍

12 小时前 · 福建

回复

喜欢

推荐阅读



已赞同 2 发表于 Paper...

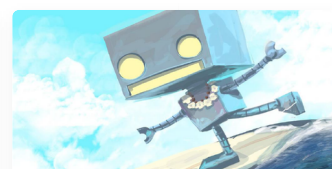
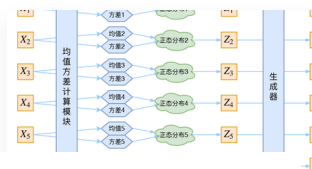


Scienc...

喜欢

分享

申请转载



科技猛兽

知乎

VAE、Diffusion、Flow Matching 系统讲解（一）：VAE 变分自编码器

首发于
[我的无人驾驶学习历程](#)